

```

1. MOVE R0, [AR0] ++ ; //Transfer
2. ADD R1, R0, R0 ; //Arithmetic operation
3. MOVE R2, [AR0] ; //Transfer
4. ADD R3, R2, R2 ; //Arithmetic operation

```

Figure 1: Sample input instructions for hypothetical processor

```

1. MOVE R0, [AR0] ++ ;
2. ADD R1, R0, R0 ; MOVE R2, [AR0] ; //ILP
4. ADD R3, R2, R2 ;

```

Figure 2: Achieving ILP in the sample instructions

Without ILP	With ILP
1. MOVE R0, [AR0] ++ ;	1. MOVE R0, [AR0] ++ ;
2. ADD R1, R0, R0 ;	2. ADD R1, R0, R0 ; MOVE R2, [AR0] ;
3. MOVE R2, [AR0] ;	4. ADD R3, R2, R2 ;
4. ADD R3, R2, R2 ;	
Instruction Cycles = 4	Instruction Cycles = 3
Code size (words) = 4	Code size (words) = 3

Figure 3: Reduction in instruction cycles and code size

Note: We also have control dependency, which involves more than one code block. However, since this article is restricted to achieving ILP within a basic block, we will not cover this dependency.

As illustrated in Figure 5, Instruction I is an instruction preceding Instruction J in the basic block.

Data flow dependency definition: An Instruction J is said to be data flow dependent on an Instruction I, if I defines an operand (variable/register) that is used by J.

For example, consider the following instructions in the example given below:

```

3. MOVE R3, [AR1] ; --> I // defines "R3"
4. ADD R4, R3, R1 ; --> J // uses "R3"

```

Here, Instruction 3 'defines' register R3, which is used as a source operand in Instruction 4 – so we have a data flow dependency between Instructions 3 and 4.

Note: Unless stated explicitly, we have assumed that in our hypothetical architecture, the first operand of an instruction is the destination operand, and the remaining are source operands.

Anti-dependency definition: An Instruction J is said to be data anti-dependent on Instruction I, if I uses an operand (variable/register) that is redefined by J.

For example, consider these instructions:

```

3. MOVE R3, [AR1] ; // "R3" defined
4. ADD R4, R3, R1 ; --> I // Uses "R3"
5. ADD R3, R4, R4 ; --> J // Redefines "R3"

```

Here, Instruction 3 'defines' register R3, which is 'used' as a source operand in Instruction 4. Then Instruction 5 redefines (or updates) R3. Thus, we have an anti-dependency between Instructions 4 and 5.

Output dependency definition: An Instruction J is said

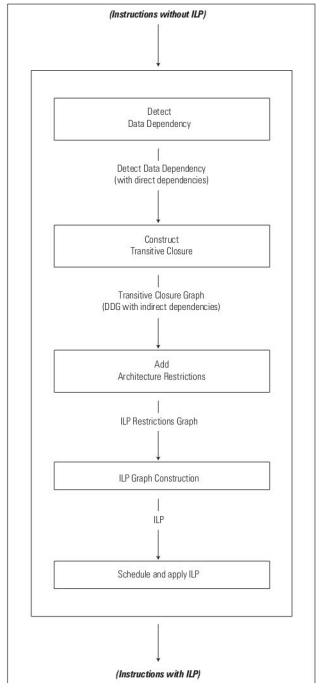


Figure 4: ILP application algorithm

to be data output-dependent on an Instruction I, if I and J re-define the same operand (variable/register).

Look at the same example given above for anti-dependency. Register R3, defined in Instruction 3, is redefined in Instruction 5. Hence, we have an output dependency between Instructions 3 and 5.

As illustrated in Figure 5, ILP is restricted when we have dependency between instructions. If an Instruction J is dependent on an Instruction I, these instructions can not be combined in parallel.